

TOHUPONO

Deterministic File-Origin Proof Infrastructure

Final Development Plan Report

v1.0 Final Planning Report | 10 July 2026

Purpose	Development blueprint for a local-first file-origin, integrity, provenance, custody, and report-signing system.
Primary beachhead	Generic legal/compliance proof workflow: contract PDF plus evidence attachment style.
Deployment default	Enterprise private/on-prem/air-gapped by design; public anchoring is optional.
Core decision	Dedicated report-signing key separate from file/manifest keys; report.pdf.sig is required.
Community Tier	Full functionality retained. Branding/footer may change only when community=True.
Status	Ready for Codex-guided terminal development and repository initialisation.

Important boundary: this system produces legal-support and evidence-bundle-ready reports. It does not guarantee court admissibility or provide legal advice. Admissibility depends on process, jurisdiction, procedure, and qualified review.

Table of Contents

- 1. Executive Summary
- 2. Strategic Decisions
- 3. Research Findings
- 4. Product Doctrine
- 5. Proof Problem
- 6. Trust Roots
- 7. Evidence Strength Model
- 8. Product Outcome
- 9. Revenue Route
- 10. Architecture
- 11. Data Model
- 12. CLI and Pro Report Workflow
- 13. Threat Model
- 14. Security and Legal Controls
- 15. Development Roadmap
- 16. Repository and Technology Stack
- 17. Testing Strategy
- 18. Deployment Strategy
- 19. Codex Operating Model
- Appendix A. AGENTS.md
- Appendix B. TODO.md
- Appendix C. Source Register

1. Executive Summary

TohuPono is a deterministic file-origin proof system for the AI era. Its purpose is to determine what can be proven about a file, not to guess whether a file is morally, historically, or legally true. The system records, signs, timestamps, verifies, packages, and reports evidence about digital artefacts such as PDFs, images, videos, audio files, software packages, datasets, AI model files, ZIP archives, and evidence bundles.

The design is local-first. Source files do not leave the device by default. Optional public anchoring can add external timestamp or transparency-log evidence, but the proof packet must remain useful in an air-gapped environment.

The strongest commercial beachhead is the generic legal/compliance workflow: a contract or formal PDF accompanied by an evidence attachment pack, manifest, verification result, chain-of-custody summary, and a signed professional report. The report must be described as legal-support/evidence-bundle ready rather than court-ready, because admissibility depends on jurisdiction, procedure, custody discipline, and qualified legal assessment.

Core Position	Decision
Category	Proof infrastructure, not an AI detector.
First product	CLI proof packet engine plus Pro PDF Report Generator.
Default deployment	Private/on-prem/air-gapped enterprise mode.
Revenue path	Paid proof reports, compliance setup, training, and enterprise deployments.
Cultural model	Kaitiaki authority support, Maori community tier, no feature restriction.
Codex role	Terminal-based assistant following AGENTS.md and TODO.md.

2. Strategic Decisions Recorded

Decision	Implementation consequence
Generic legal/compliance workflow prioritised	The first practical workflow is contract PDF plus evidence attachments, proof packet, signed report, and verification command.
Enterprise private deployment is the design default	The system must run without public cloud, SaaS dependency, or live verification services unless explicitly configured.
Dedicated report-signing key implemented	The PDF report gets its own signing key and detached signature, separate from file/manifest keys.
Community Tier has full functionality	Passing community=True changes banner/footer only. It must not disable features.
Truth claims are constrained	The system classifies evidence strength and conflict. It does not declare universal authenticity.

3. Research Findings and Development Implications

Research area	Finding	Implication for TohuPono
C2PA / Content Credentials	C2PA provides signed provenance manifests, assertions, claims, signer identities, time-stamps, validation states, and embedding/sidecar options. [S1][S2]	Use C2PA as a supported evidence layer, not the whole product.
C2PA limitations	Recent 2026 research warns that C2PA alone should not be relied upon prematurely for high-stakes uses such as journalism, finance, or legal evidence. [S3]	Use cross-layer verification, contradiction checks, and careful verdict language.
Sigstore/cosign/Rekor	Cosign signs artefacts and Rekor provides tamper-resistant transparency-log metadata for software supply chains. [S4][S5]	Use as optional software/package/model evidence layer.

OpenTimestamps	Provides blockchain timestamp proof that data existed before a point in time, with local hash-first privacy. [S6]	Use as optional public proof-of-existence layer.
RFC 3161 TSA	Defines Time-Stamp Authority request/response structures and TSA requirements. [S9]	Add enterprise-compatible timestamp authority integration later.
SLSA	Provides an industry specification for improving software supply-chain security. [S7]	Use SLSA/in-toto style attestations for software, build, model, and dataset artefacts.
BagIt	Provides a file packaging format with payload manifests and checksums for transfer and preservation. [S8]	Use BagIt export for archive/evidence bundle transport.
Post-quantum signatures	NIST finalised ML-DSA and SLH-DSA digital signature standards in 2024. [S10][S11]	Design key/signature abstractions now; add PQ implementations behind feature flags when libraries mature.

4. Product Doctrine

TohuPono must be evidence-first. A missing watermark, missing metadata, missing C2PA manifest, or missing timestamp does not prove fakery. It proves absence of that evidence. Conversely, a valid signature proves that a key signed a claim, not that the real-world event described by the file is true.

- Hash proves file identity.
- Hash comparison proves integrity or alteration relative to a recorded digest.
- Timestamp proves existence before a time, not authorship.
- Signature proves a key made a claim, subject to trust policy.
- C2PA proves a signed provenance structure when valid, not perfect truth.
- Transparency logs prove publication/registration of metadata, not real-world truth.
- Chain-of-custody proves recorded handling events when the chain is complete and signed.
- Verdicts must expose evidence and limitations.

Unsafe wording	Required wording
Court-ready	Legal-support/evidence-bundle ready.
Real	Verified integrity, verified origin chain, or timestamped existence.
Fake	Unproven, suspicious, provenance conflict, altered after proof, or fake by contradiction.
Not AI-generated	No AI-generation evidence found, or C2PA/watermark status as observed.
Impossible to forge	Tamper-evident under stated cryptographic and trust assumptions.

5. Define the Proof Problem

5.1 First-priority files

Priority	File class	Reason
1	PDFs, office documents, contracts, invoices, evidence attachments	Highest legal/compliance and commercial value; aligns with Pro Report workflow.
1	Images, screenshots, video, audio	High AI-forgery and public-trust risk; aligns with C2PA and capture provenance.
2	ZIP, source archives, APKs, containers, model files, datasets	Strong supply-chain relevance; aligns with Sigstore, SLSA, and Rekor.
3	Archival folders and mixed evidence bundles	Strong preservation value; aligns with BagIt and chain-of-custody.

5.2 Who must believe the proof

- Individual creators and file owners need simple integrity and timestamp proof.
- Commercial parties need signed reports, repeatable verification, and audit trails.
- Legal-support teams need custody discipline, disclaimers, report signatures, and reproducible outputs.
- Archives need fixity, export packaging, and long-term verification.
- Maori organisations need kaitiaki authority support, consent metadata, protected records, and offline control.
- Public users need a later web verifier or QR proof page that exposes only approved fields.

5.3 Proof locality

The correct model is local-first with optional public anchoring. The proof packet must be fully useful offline. Public anchoring is optional and should publish hashes, signatures, or metadata only when the user explicitly chooses it. Sensitive taonga, legal records, private contracts, or whakapapa-related material must not be exposed to public infrastructure by default.

5.4 Proof targets

Target	Can TohuPono prove it?	Evidence required
File identity	Yes	Cryptographic digest.
Integrity since sealing	Yes	Prior digest compared against current digest.
Existence before a time	Yes	OpenTimestamps, RFC 3161, or trusted timestamp evidence.
Signer claim	Yes, within trust policy	Valid signature, public key/certificate, signer status.
Authorship	Partly	Signature plus trusted identity plus process evidence.
Origin chain	Partly to strongly	Manifest, signature, timestamp, device/source identity, custody events.
Chain of custody	Yes if recorded correctly	Append-only signed events with no unexplained gaps.
Real-world truth	No	Requires external investigation beyond file proof.

6. Define Trust Roots

Trust root	Purpose	Default status
User key	Personal signing and authorship claim.	Enabled for local proof.
Organisation key	Company, trust, compliance, or publisher proof.	Enterprise configuration.
Device key	Capture device, scanner, phone, camera, or server proof.	Planned for Android/desktop capture.
Kaitiaki key	Maori authority/custodian approval for protected records.	Optional sovereign authority layer.
Witness key	Multi-party corroboration and dispute-resistant custody.	Planned.
Report-signing key	Signs generated PDF reports only.	Required for Pro tier.
Timestamp authority	Time proof.	Optional: OTS/RFC 3161.
Transparency log	Public auditability.	Optional: Rekor or private log.
C2PA certificate chain	Media provenance verification.	Optional evidence plugin.
Post-quantum key	Long-lived future-resistant proof.	Roadmap, feature gated.

Revocation must be append-only. Proofs should not be deleted; they should be superseded, revoked, disputed, or marked compromised. A lost key is different from a compromised key. Historical proofs created before a key was lost may remain valid if compromise is not proven and timestamp evidence supports the signing time.

7. Evidence Strength Model

Level	Evidence class	Strength	Meaning
0	No evidence	None	The file is unproven.
1	Hash only	Basic	Exact file identity is known.
2	Hash + metadata	Basic+	Observed properties were recorded.
3	Hash + timestamp	Moderate	File or digest existed before a time.
4	Hash + signature	Strong	A key signed this exact digest or manifest.
5	Hash + signature + timestamp	Stronger	A signer claim existed before a known time.
6	Provenance manifest	Contextual	Creation/edit/capture assertions can be validated.
7	Transparency log entry	Auditable	Signed metadata was publicly or privately logged.
8	Chain-of-custody trail	Legal/archival	Handling events are continuous and signed.
9	Multi-party witness + PQ-ready signature	High assurance	Multiple authorities and long-term crypto migration path.

Verdict	Meaning
VERIFIED_INTEGRITY	The current file digest matches the sealed digest.
ALTERED_AFTER_PROOF	The current digest does not match the sealed digest.
TIMESTAMPED_EXISTENCE	The digest has valid timestamp proof.
SIGNED_CLAIM_ONLY	A signer made a claim, but trust/authority may be limited.
VERIFIED_ORIGIN_CHAIN	Signatures, timestamps, manifests, and policy align.
CUSTODY_VERIFIED	Custody trail is complete under policy.
PROVENANCE_CONFLICT	Valid evidence signals conflict.
UNPROVEN	Insufficient evidence.
SUSPICIOUS	Signals suggest concern but do not prove contradiction.
FAKE_BY_CONTRADICTION	A hard claim is contradicted by hard evidence.

8. Product Outcome

Stage	Product	Purpose
1	Command-line tool	Proof, verify, inspect, attest, timestamp, report.
1	PDF report service	Fastest revenue path and legal/compliance beachhead.
2	Offline vault	Local proof storage, search, import/export, audit trail.
2	Desktop app	Drag-and-drop proofing and report preview.
3	Android app	Capture-time hashing, field proof, QR proof export.
3	Web verifier	Public/private proof check without exposing source files.
4	Hosted portal/API	Organisation dashboards, batch verification, integrations.
5	White-label sovereign proof system	Private deployments for organisations, iwi, hapu, marae, archives, and enterprises.

9. Revenue Route

The first revenue route should be paid proof reports and compliance setup, because this can begin before SaaS infrastructure exists. Manual or semi-automated report services will reveal real user needs and convert repeated tasks into product features.

Revenue route	Start stage	Why
Paid proof reports	Immediate	Fastest path from CLI/report generator to income.
Consulting/setup	Immediate	Enterprise and legal/compliance users need configuration and training.
Training/workshops	Immediate	Good fit for creators, Maori organisations, and evidence-handling teams.
Software licences	After MVP	Requires stable CLI, docs, installers, and support model.
Hosted proof vaults	After security hardening	Recurring revenue but requires privacy, hosting, compliance, and support.
Verification API	After adoption	Best when repeated verification volume exists.
White-label systems	Later	Highest value after proven architecture and pilots.

10. System Architecture

TohuPono should use a composed-core architecture. The value is in the proof packet, verdict engine, trust policy, report generator, custody workflow, and offline deployment discipline. Existing standards provide specialised evidence layers.

```
tohupono/  
cli.py  
core/  
  hashing.py  
  file_identity.py  
  canonical_json.py  
  metadata.py  
evidence/  
  c2pa_adapter.py  
  sigstore_adapter.py  
  timestamps.py  
  bagit_export.py  
  custody.py  
reporting/  
  pro_report.py  
  templates.py  
  report_signing.py  
trust/  
  keys.py  
  policy.py  
  revocation.py  
  signers.py  
verdicts/  
  classifier.py  
  evidence_strength.py
```

```

language.py
vault/
  sqlite_store.py
  append_log.py
api/
desktop/
android/
docs/
tests/

```

Layer	Responsibility
File identity engine	Digest calculation, file size, magic-byte identification, observed metadata.
Metadata extraction	EXIF, PDF metadata, document properties, media codec metadata, APK/package metadata.
Evidence adapters	C2PA, Sigstore/cosign, Rekor, OpenTimestamps, RFC 3161, SLSA, BagIt.
Trust policy	Trusted keys, authority roles, key status, revocation, dispute policy.
Verdict engine	Evidence classification, conflict detection, confidence language.
Report engine	Markdown/PDF reports, legal-support disclaimer, report.pdf.sig.
Vault	Offline proof storage, append-only audit events, import/export.
Interfaces	CLI first, then desktop, Android, web verifier, hosted API.

11. Data Model

The proof packet is the central artefact. It must be portable, deterministic, inspectable, and verifiable without a hosted account. Sensitive deployments can store only hashes and structured evidence, not payload files.

```

proof_packet/
  manifest.json
  hashes.txt
  metadata.json
  evidence_chain.jsonl
  verdict.json
  verification_report.md
  warnings.json
  signatures/
    manifest.sig
    manifest.pub
  timestamps/
    file.ots
    rfc3161.tsr
  c2pa/
    c2pa_manifest.json
  bagit/

```

11.1 Manifest schema v0.1

```

{
  "schema_version": "tohupono.proof_manifest.v0.1",

```



```

"proof_id": "tp_...",
"created_at_utc": "2026-07-10T00:00:00Z",
"tool": {
  "name": "tohupono",
  "version": "0.1.0"
},
"file": {
  "path_observed": "contract.pdf",
  "name": "contract.pdf",
  "size_bytes": 123456,
  "mime_observed": "application/pdf",
  "sha256": "...",
  "sha512": "..."
},
"claims": [
  {
    "claim_id": "claim_001",
    "type": "received_from_user",
    "statement": "User supplied file for proof sealing.",
    "issuer": "local_user",
    "evidence_refs": []
  }
],
"evidence": {
  "hashes": [],
  "timestamps": [],
  "signatures": [],
  "c2pa": [],
  "transparency_logs": [],
  "custody_events": []
},
"trust_policy": {
  "policy_id": "local_default",
  "policy_digest": "sha256:..."
},
"warnings": [],
"community": false
}

```

12. CLI and Pro Report Workflow

The CLI is the first product surface. It must expose the proof workflow in direct commands that can be used by Codex in terminal, by power users, and later by desktop/API wrappers.

Create proof packet

tohupono prove ./contract.pdf --output proof_packet/

Verify proof packet against current file

tohupono verify ./contract.pdf --proof proof_packet/manifest.json --output verification_report.md

```
# Generate professional PDF report and detached report signature
tohu pono report --proof proof_packet/manifest.json --format pdf --output verification_report.pdf

# Generate full-feature community-branded report
tohu pono report --proof proof_packet/manifest.json --format pdf --output community_report.pdf --community
```

12.1 Pro Report Generator requirements

- Prominent colour-coded classification header.
- File identity table: path, SHA-256, size, sealed timestamp.
- Evidence summary: reasons, conflicts, missing signals.
- Signature and trust validation section.
- Chain-of-custody summary.
- Legal-support/evidence-bundle disclaimer.
- Community banner/footer when community=True.
- Detached cryptographic signature over the PDF itself using dedicated report-signing key.

13. Threat Model

Threat	Control
File altered after proof	Digest mismatch produces ALTERED_AFTER_PROOF.
Metadata stripped	Record METADATA_STRIPPED or metadata_absent; do not call fake by default.
False authorship claim	Require trusted signature and policy; classify as signed claim if not independently trusted.
Forged proof packet	Canonical JSON plus manifest signature plus trust policy verification.
Report tampering	Detached report.pdf.sig from dedicated report key.
Key loss	Mark lost, preserve historical proof if timestamp and policy support it.
Key compromise	Mark compromised; affected proofs become disputed or invalid under policy.
Public anchor leaks sensitive data	Anchor hashes only; public anchoring opt-in; clear privacy warning.
C2PA over-trust	Treat C2PA as one evidence layer; inspect conflicts with watermark, metadata, and custody evidence.
AI detector false positive/negative	Do not rely on detector output as final truth.
Supply-chain tampering	Use Sigstore/SLSA/Rekor for software artefacts.
Insider custody gap	Signed append-only custody events and mandatory gap reporting.

14. Security, Legal and Cultural Controls

14.1 Security controls

- Dedicated key purposes: manifest, file, report, timestamp authority, witness, organisation, kaitiaki.
- Restrictive permissions for private keys.
- Fail-closed verification for invalid hashes, signatures, or policy errors.
- Opt-in network calls only.
- Append-only custody events.
- No raw sensitive file content in logs.
- JSON schema validation for proof packets.
- Stable canonical JSON for deterministic hashing/signing.

14.2 Legal-support boundary

Reports should state that they summarise technical evidence and are suitable for legal-support/evidence-bundle preparation. They must not guarantee admissibility, legal sufficiency, or court acceptance. A court, tribunal, regulator, lawyer, or qualified evidence practitioner may require additional custody procedures and sworn statements.

14.3 Maori authority model

The Maori-aligned model adds authority without weakening technical neutrality. The system can support kaitiaki keys, community-branded reports, protected metadata, consent fields, taonga classifications, and offline vaults for iwi, hapu, marae, trusts, and Maori service providers. These fields must never be decorative only; they must map to authority, access, consent, custody, or reporting behaviour.

- Tikanga: disciplined evidence practice and correct boundaries.
- Whakapapa: origin, lineage, custody, and relationship of file evidence.
- Tino rangatiratanga: local control, self-determined trust roots, no forced cloud dependency.
- Kaitiaki: protected handling of cultural records and authority-controlled disclosure.

15. Development Roadmap

Phase	Weeks	Deliverables
0. Repository foundation	0-1	Package structure, pyproject, CLI shell, tests, README, AGENTS.md, TODO.md.
1. File identity engine	1-2	Hashing, file info, canonical JSON, inspect/hash commands.
2. Proof packet v0.1	2-4	manifest.json, evidence_chain.jsonl, metadata.json, warnings.json, prove command.
3. Verification engine	4-5	Digest comparison, verdict JSON/MD, altered-file tests.
4. Pro report integration	5-6	report command, PDF generation, report.pdf.sig, community flag.
5. Signing and trust policy	6-8	Keygen, manifest signing, trust policy YAML, revocation states.
6. Timestamp support	8-10	OpenTimestamps optional integration, RFC 3161 adapter interface.
7. Evidence integrations	10-12	C2PA inspection, BagIt export, SLSA/Sigstore planning.
8. Enterprise pilot	12+	Offline vault, private deployment guide, compliance pilot, report pricing.

15.1 Immediate next slice

1. Move ``pro_report.py`` into ``tohu pono/reporting/pro_report.py`` or wrap it cleanly.
2. Add ``tohu pono report`` command.
3. Add dedicated report key generation on first use.
4. Generate ``verification_report.pdf`` and ``verification_report.pdf.sig``.
5. Add ``--community`` flag with banner/footer only.
6. Add tests covering report generation, signature existence, key separation, and community output.
7. Update README and examples.

16. Repository and Technology Stack

Area	Recommended choice
MVP language	Python 3.11+ for fastest iteration and current <code>`pro_report.py`</code> compatibility.
CLI framework	Typer or argparse. Typer preferred for clean subcommands.
Schema validation	Pydantic or dataclasses plus jsonschema.
Cryptography	cryptography / PyNaCl / age-compatible libs where suitable; no custom crypto.
PDF reports	ReportLab initially; optional DOCX/PDF template layer later.
Metadata	python-magic, exiftool adapter, pypdf/PyMuPDF, filetype, mutagen where needed.

C2PA	External `c2patool` or c2pa-rs adapter first.
Timestamping	OpenTimestamps client, RFC 3161 adapter later.
BagIt	bagit-python or native minimal BagIt writer.
Vault	SQLite with append-only event table.
API	FastAPI later.
Desktop	Tauri later, wrapping CLI/core.
Android	Kotlin later, shared proof packet schema.

17. Testing Strategy

- Unit tests for hashing, canonical JSON, schema validation, verdict classification, and key handling.
- Integration tests for prove -> verify -> report.
- Golden fixtures to ensure proof packet stability.
- Negative tests for altered files, missing proofs, invalid signatures, corrupted reports, revoked keys, and missing timestamps.
- Offline tests proving no network call is made by default.
- Report tests confirming PDF and `.sig` are generated and community flag does not restrict features.
- Security tests checking private key permissions and overwrite refusal.
- Regression tests for terminology: avoid unsupported “court-ready”, “real”, “not AI”, and absolute authenticity claims.

python -m compileall tohupono tests

pytest -q

ruff check .

mypy tohupono

18. Deployment Strategy

Deployment mode	Description
Local personal	CLI on user machine; local proof packets only.
Community	Full functionality with community=True branding/footer for Maori sovereignty, iwi, hapu, marae use.
Professional	Signed PDF reports, evidence bundles, report key management.
Enterprise on-prem	Private server, internal trust roots, internal timestamp authority, local vault.
Air-gapped	No network by default; import/export proof packets via controlled media.
Hosted verifier	Later public/private proof checking with strict privacy controls.
API	Batch verification and integration after stable core.

19. Codex Operating Model

Codex should be treated as the terminal development assistant. It should read AGENTS.md and TODO.md, inspect the repository, apply small patches, run tests, and report exactly what changed. Codex must not redesign the architecture casually, invent cryptography, or weaken legal/authenticity wording.

- Codex starts with repository inspection, not assumptions.
- Codex chooses the smallest incomplete TODO item.
- Codex writes tests with each feature.
- Codex runs local validation commands.
- Codex updates documentation.
- Codex summarises changes, tests, and remaining gaps.

Appendix A. AGENTS.md

AGENTS.md - Codex Operating Guide for TohuPono

Mission

Build TohuPono as a deterministic, local-first file-origin proof system. The product must generate, verify, package, sign, timestamp, and report evidence about digital files without making unsupported claims about real-world truth.

The system proves evidence state: file identity, integrity, timestamped existence, signer claims, provenance signals, chain-of-custody continuity, and contradictions between claims and evidence.

Non-negotiable principles

1. Do not invent custom cryptography.
2. Do not claim a file is "real", "true", "not AI", or "court-proof" unless the evidence model explicitly supports the narrower claim.
3. Default to offline/private operation. Network anchoring is opt-in.
4. Treat provenance as layered evidence, not a single score.
5. Keep the report-signing key separate from file-signing and manifest-signing keys.
6. Preserve chain-of-custody as append-only events.
7. Make all output deterministic unless a random key, nonce, timestamp, or signature is explicitly part of the operation.
8. Never log private keys, secrets, raw sensitive file content, or private metadata by default.
9. Prefer open standards and well-reviewed libraries: C2PA, Sigstore/cosign, Rekor, OpenTimestamps, RFC 3161, SLSA, BagIt, NIST PQ signatures.
10. Community Tier must remain fully functional. Branding may differ; features must not be restricted.

Architecture guardrails

- Core logic lives in `tohupono/core/` and must be UI-independent.
- CLI code only orchestrates core functions. Do not bury business logic in command handlers.
- Evidence integrations live under `tohupono/evidence/`.
- Trust policy and key handling live under `tohupono/trust/`.
- Report generation lives under `tohupono/reporting/`.
- Verdict classification lives under `tohupono/verdicts/`.
- Vault and persistence live under `tohupono/vault/`.
- Tests must cover both success and failure paths.

Required initial commands for Codex

Before editing, run:

```
pwd
find . -maxdepth 3 -type f | sort
python --version || true
pytest --version || true
```

If this is a new repo, initialise the structure from TODO.md before implementing features.

Development workflow

1. Read `TODO.md` and choose the smallest incomplete task.
2. Inspect existing code before editing.
3. Make one coherent change at a time.
4. Add or update tests with every behaviour change.
5. Run the relevant tests.
6. Update documentation and examples.
7. Summarise what changed, what was tested, and what remains.

Coding standards

- Python target: 3.11+.
- Use type hints for public functions.
- Use `pydantic` or dataclasses for structured data.
- Use canonical JSON: UTF-8, sorted keys, stable separators.
- Use explicit exceptions with actionable messages.
- Avoid global mutable state.
- Keep functions small and testable.
- Use clear enum values for verdicts and evidence classes.
- Use British English in user-facing documentation.
- Use te reo Maori names only where they clarify project meaning. Do not use macrons in filenames, package names, or identifiers unless explicitly requested.

Security rules

- Private keys must be created with restrictive file permissions.
- Report-signing keys must be separate from file and manifest keys.
- CLI must refuse to overwrite existing keys unless `--force` is explicitly supplied.
- Never upload files or proof packets unless the user selected an anchoring or hosted mode.
- External calls must be behind explicit flags: `--ots`, `--rekor`, `--c2pa`, `--public-anchor`, or similar.
- Verification must fail closed when signature, hash, policy, or timestamp validation is ambiguous.
- Missing evidence is `UNPROVEN`, not `FAKE`.
- Contradictory evidence is `PROVENANCE\_CONFLICT` or `FAKE\_BY\_CONTRADICTION` only where hard evidence contradicts a claim.

Testing expectations

Every milestone needs:

- unit tests for pure functions;
- integration tests for CLI flows;
- golden fixture tests for proof packet stability;
- negative tests for altered files, bad signatures, missing keys, invalid manifests, and revoked/disputed proofs;
- report generation tests ensuring PDF and detached `.sig` are created.

Minimum test command:

```
pytest -q
```

Recommended local quality gate:

```
python -m compileall tohupono tests
pytest -q
ruff check . || true
mypy tohupono || true
```

Documentation expectations

Update documentation whenever behaviour changes:

- README quickstart;
- CLI help examples;
- proof packet schema notes;
- trust policy schema;
- verdict definitions;
- legal-support disclaimer;
- enterprise/offline deployment notes.

Commit discipline

Use small commits. Suggested format:

```
feat(cli): add report command
fix(verify): fail closed on missing manifest digest
test(report): cover detached PDF signature generation
docs(trust): document report-signing key separation
```

Codex should propose commit messages but must not assume commits were made unless it actually ran git commit.

Definition of done

A task is complete only when:

- implementation exists;
- tests pass;
- documentation is updated;
- CLI usage is demonstrated;
- outputs are deterministic where expected;
- security constraints are preserved;
- user-facing language avoids unsupported legal or authenticity claims.

Appendix B. TODO.md

TODO.md - Codex Guided Development Plan for TohuPono

Current target

Integrate the Pro Report Generator into the main CLI and produce an end-to-end proof workflow:

```
tohupono prove ./file.pdf --output proof_packet/  
tohupono verify ./file.pdf --proof proof_packet/manifest.json  
tohupono report --proof proof_packet/manifest.json --format pdf --output verification_report.pdf --community false
```

The report command must create:

```
verification_report.pdf  
verification_report.pdf.sig
```

The `.sig` must be produced with a dedicated report-signing key, not the file proof key.

Milestone 0 - Repository foundation

Tasks

- Create Python package structure.
- Add `pyproject.toml`.
- Add CLI entry point `tohupono`.
- Add README, AGENTS.md, TODO.md, LICENSE placeholder, SECURITY.md.
- Add tests folder with smoke test.

Acceptance criteria

- `python -m tohupono --help` works.
- `pytest -q` runs.
- Package imports cleanly.

Suggested structure

```
tohupono/  
  __init__.py  
  __main__.py  
  cli.py  
  core/  
  evidence/
```


reporting/
trust/
verdicts/
vault/
tests/
docs/
examples/
keys/.gitkeep

Milestone 1 - Deterministic file identity engine

Tasks

- Implement SHA-256, SHA-512, and BLAKE3 where available.
- Detect file size.
- Detect MIME by magic bytes where possible.
- Record filesystem timestamps as observed metadata, not proof of origin.
- Create canonical JSON helper.

CLI

```
tohupono inspect ./file.pdf  
tohupono hash ./file.pdf --algorithm sha256
```

Acceptance criteria

- Same file produces same digest on repeated runs.
- Extension mismatch is warned, not fatal.
- Tests cover empty file, binary file, renamed extension, and missing file.

Milestone 2 - Proof packet v0.1

Tasks

- Create `ProofManifest` schema.
- Create proof packet directory layout.
- Add manifest writing.
- Add evidence chain JSONL with first `sealed` event.
- Add warnings file.

Output

```
proof_packet/  
  manifest.json  
  hashes.txt  
  metadata.json  
  evidence_chain.jsonl  
  warnings.json
```

CLI

```
tohu pono prove ./file.pdf --output proof_packet/
```

Acceptance criteria

- Proof packet validates against schema.
- Manifest has stable field order.
- Hashes match the source file.
- No source file content is copied unless `--include-payload` is provided.

Milestone 3 - Verification engine

Tasks

- Load manifest.
- Recalculate current file digest.
- Compare against manifest digest.
- Evaluate evidence classes.
- Produce verdict JSON and Markdown summary.

CLI

```
tohu pono verify ./file.pdf --proof proof_packet/manifest.json --output verification_report.md
```

Verdicts

- `VERIFIED\_INTEGRITY`
- `ALTERED\_AFTER\_PROOF`
- `UNPROVEN`
- `SIGNED\_CLAIM\_ONLY`
- `TIMESTAMPED\_EXISTENCE`
- `PROVENANCE\_CONFLICT`

- `FAKE\_BY\_CONTRADICTION`

Acceptance criteria

- Original file verifies as `VERIFIED\_INTEGRITY`.
- Modified file verifies as `ALTERED\_AFTER\_PROOF`.
- Missing evidence is `UNPROVEN`, not `FAKE`.

Milestone 4 - Pro Report Generator integration

Tasks

- Move or wrap `pro\_report.py` under `tohu pono/reporting/pro\_report.py`.
- Add CLI command `tohu pono report`.
- Add `--proof`, `--format`, `--output`, `--community`, and `--report-key` options.
- Generate a dedicated report signing key on first use when not supplied.
- Create detached `report.pdf.sig`.
- Include legal-support/evidence-bundle disclaimer.

CLI

```
tohu pono report --proof proof_packet/manifest.json --format pdf --output verification_report.pdf
```

Community Tier:

```
tohu pono report --proof proof_packet/manifest.json --format pdf --output community_report.pdf --community
```

Acceptance criteria

- PDF report exists.
- Detached `.sig` exists.
- Report key is separate from manifest/file keys.
- Community report has banner/footer but no feature restrictions.
- Tests verify all above.

Milestone 5 - Signing and trust policy

Tasks

- Add key generation command.

- Add manifest signing.
- Add signature verification.
- Add trust policy YAML.
- Add key status: active, revoked, lost, compromised, retired.

CLI

```
tohuono keygen --purpose manifest --out keys/manifest_key.pem
tohuono keygen --purpose report --out keys/report_signing_key.pem
tohuono attest --proof proof_packet/manifest.json --key keys/manifest_key.pem
```

Acceptance criteria

- Signature verifies against public key.
- Wrong key fails verification.
- Revoked key fails for new proofs.
- Proofs created before loss date can remain historically valid unless compromise is declared.

Milestone 6 - Timestamp support

Tasks

- Add RFC 3161-ready interface.
- Add OpenTimestamps optional integration.
- Store timestamp evidence separately.
- Verify timestamp proofs where available.

CLI

```
tohuono timestamp --proof proof_packet/manifest.json --ots
tohuono verify ./file.pdf --proof proof_packet/manifest.json --check-timestamps
```

Acceptance criteria

- No network call occurs unless timestamp option is selected.
- Pending OpenTimestamps state is represented clearly.
- Failed timestamp validation is a warning or failure according to policy.

Milestone 7 - C2PA media provenance

Tasks

- Add optional c2patool/c2pa-rs adapter.
- Extract C2PA manifests where present.
- Validate C2PA status and record validation results.
- Detect absence or stripping of C2PA as `metadata\_absent`, not fake.

CLI

```
tohupono inspect ./image.jpg --c2pa
tohupono verify ./image.jpg --proof proof_packet/manifest.json --c2pa
```

Acceptance criteria

- Supported file with C2PA reports signer, claim, validation status.
- Unsupported file gives clear message.
- C2PA conflict becomes `PROVENANCE\_CONFLICT`.

Milestone 8 - BagIt archival package

Tasks

- Add BagIt export.
- Include proof packet and optional payload.
- Generate manifest-sha256.txt.
- Validate bag integrity.

CLI

```
tohupono bag --proof proof_packet/manifest.json --payload ./file.pdf --output sealed_bag/
tohupono bag-verify sealed_bag/
```

Acceptance criteria

- Bag validates after creation.
- Removing a file fails validation.
- Changing payload fails validation.

Milestone 9 - Enterprise offline vault

Tasks

- Add local SQLite vault.
- Add append-only events table.
- Add import/export proof packet.
- Add offline policy mode.
- Add air-gapped deployment guide.

CLI

```
tohuono vault init
tohuono vault import proof_packet/
tohuono vault search --sha256 <digest>
```

Acceptance criteria

- Vault works without internet.
- Vault export produces portable proof packet.
- Audit events are append-only.

Milestone 10 - API and web verifier

Tasks

- Add FastAPI verification service.
- Add public hash lookup endpoint.
- Add proof upload endpoint for verification only.
- Add QR proof summary model.
- Add authentication for private/hosted mode.

Acceptance criteria

- Local API verifies file + proof.
- Hosted mode does not store source file unless configured.
- Public verifier exposes only approved fields.

Milestone 11 - Desktop and Android companion

Desktop

- Build later using Tauri or Electron shell around the CLI/core library.

- Drag-and-drop file verification.
- Report preview.
- Key management UI.

Android

- Capture-time hashing.
- Local proof packet creation.
- Optional device key integration.
- QR proof export.
- Offline-first design.

Acceptance criteria

- Both clients rely on the same core evidence model.
- No separate verdict logic is implemented in UI clients.

Milestone 12 - Post-quantum roadmap

Tasks

- Add algorithm registry abstraction.
- Add metadata fields for ML-DSA and SLH-DSA readiness.
- Test PQ signatures using experimental libraries only behind feature flags.
- Do not make PQ signatures mandatory until deployment libraries are mature.

Acceptance criteria

- Classical signatures remain supported.
- PQ fields are represented cleanly in manifests.
- Verification fails clearly if PQ verifier is unavailable.

Immediate Codex task order

1. Create or inspect repo structure.
2. Add/confirm `tohupono` CLI entrypoint.
3. Add `report` subcommand.
4. Move/wrap `pro\_report.py` into package.
5. Implement report key generation if missing.
6. Generate PDF and `.sig` from `manifest.json`.
7. Add tests for pro report generation.

8. Update README examples.
9. Run test suite.
10. Summarise results.

Final release gate for MVP

- `tohupono prove` works.
- `tohupono verify` works.
- `tohupono report` works.
- PDF report and detached signature are created.
- Original file and altered file produce different verdicts.
- Community Tier is fully functional.
- Enterprise/offline mode is documented.
- No unsupported legal or authenticity claims remain in user-facing output.

Appendix C. Source Register

The following sources informed the architecture, evidence model, and implementation roadmap. Source references are included for traceability and should be refreshed before major public release.

ID	Source	URL
S1	C2PA Technical Specification 2.2	https://spec.c2pa.org/specifications/specifications/2.2/specs/C2PA_Specification.html
S2	C2PA Rust SDK, contentauth/c2pa-rs	https://github.com/contentauth/c2pa-rs
S3	Golaszewski et al., Verifying Provenance of Digital Media: Why the C2PA Specifications Fall Short, 2026	https://arxiv.org/abs/2604.24890
S4	Sigstore cosign repository	https://github.com/sigstore/cosign
S5	Sigstore Rekor logging overview	https://docs.sigstore.dev/logging/overview/
S6	OpenTimestamps	https://opentimestamps.org/
S7	SLSA Specification v1.2	https://slsa.dev/spec/v1.2/
S8	RFC 8493, The BagIt File Packaging Format V1.0	https://datatracker.ietf.org/doc/html/rfc8493
S9	RFC 3161, Internet X.509 Public Key Infrastructure Time-Stamp Protocol	https://datatracker.ietf.org/doc/html/rfc3161
S10	NIST FIPS 204, Module-Lattice-Based Digital Signature Standard	https://csrc.nist.gov/pubs/fips/204/final
S11	NIST FIPS 205, Stateless Hash-Based Digital Signature Standard	https://csrc.nist.gov/pubs/fips/205/final

End of report.